

Passing the Test You Trained On: Re-evaluating Prompt-Injection Detectors for LLM Agents

Anonymous submission

Abstract

LLM agents increasingly screen tool outputs with small prompt-injection detectors, and teams choose among detectors by their scores on public benchmarks. We ask whether those scores predict how a detector behaves inside an agent. We replay the ground-truth tool calls of two agent benchmarks, AgentDojo and τ -bench, without an LLM to obtain tool outputs that are benign by construction, label injected outputs by differential replay, and evaluate fifteen detectors, including Meta’s Prompt Guard 2, and two task-aware LLM judges on these outputs and on the BIPIA benchmark. Detection rankings transfer poorly between benchmarks: the best detector on BIPIA catches 2% of AgentDojo injections at a 1% false-positive rate, and a detector that catches 72% of AgentDojo injections catches 15% on τ -bench. False-positive rates on tool outputs, which range from none to over 90%, do transfer between the two agent benchmarks. Where training data is public, the form of the training inputs explains the results. The BIPIA leader was trained on full BIPIA inputs, but having seen InjecAgent’s attack strings as short prompts does not help it find them inside tool outputs; the best detector on both agent benchmarks shares no data with any benchmark and was trained on agent-style inputs. Evaluations meant to inform deployment should use the agent’s own tool outputs, report detection at a low false-positive rate, and audit what the detector was trained on.

1 Introduction

LLM agents read email, calendars, web pages, and transaction histories on a user’s behalf, and every one of those sources can carry text written by someone else. Indirect prompt injection turns that text into instructions [11]. The cheapest defense to deploy is a detector: a small classifier that screens every tool output before the agent sees it and drops what looks like an injection [5, 15, 21]. It needs no change to the agent or the model, which is why guardrail toolkits ship detectors and practitioners compare them.

The threat is simple to state. An attacker who can write a calendar event description, a product review, or a table cell controls part of what a tool returns; if the detector misses that text, the agent may act on it. If the detector is too eager instead, it drops ordinary tool outputs, and the agent cannot finish the user’s task.

How would a team decide which detector to deploy? Detector papers and third-party comparisons report accuracy on prompt collections and injection benchmarks; a 2025 evaluation of nine open guardrails, for example, concluded from BIPIA’s email and table tasks that PIGuard detects indirect injection effectively [18]. Two things in this practice are unchecked. First, benchmark text is not what an agent’s detector screens. Agents see serialized tool outputs (records, lists, confirmations), and false alarms on those translate into failed tasks, not into a percentage on a balanced test set. Second, nobody checks whether the benchmark overlaps with the detector’s training data. Prior work showed that same-source splits inflate scores for classifiers the authors train themselves [9] and argued for low-FPR evaluation [13]; neither audits released detectors or evaluates them where an agent would use them.

We evaluate fifteen detectors, from 2023 DeBERTa classifiers through Meta’s Prompt Guard 2 to 2026 models built for agent inputs, together with two task-aware LLM judges, on the tool outputs of two agent benchmarks, AgentDojo [7] and τ -bench [23], and on BIPIA [24]. Our central observation is that a detector works on inputs whose form resembles what it was trained on, and that benchmark scores mostly measure this resemblance rather than a general ability to detect injections.

Making this measurement requires three things that existing benchmarks do not provide. First, benign agent tool outputs with certain ground truth: we replay the benchmarks’ ground-truth tool calls without any LLM, which yields tool outputs that are benign by construction. Second, reliable labels for injected outputs: substring matching fails because AgentDojo re-wraps injected text when it serializes outputs, so we label by differential replay against the clean environ-

ment. Third, a way to connect performance to training data: we audit the published training data of two detectors against all three benchmarks and against InjecAgent’s attacks [25], and we include a judge whose training data predates AgentDojo.

The results reverse the conclusions a benchmark-based comparison would draw. Detection rankings transfer poorly: between BIPIA and AgentDojo Kendall’s τ is 0.01, and even between the two agent benchmarks it is 0.28. PIGuard catches 95.1% of BIPIA injections at 1% FPR and 2.1% of AgentDojo injections; Prismor catches 72.2% of AgentDojo injections and 15.2% of τ -bench’s. False-positive rates on tool outputs, by contrast, range from none to over 90% and are consistent across the two agent benchmarks ($\tau = 0.67$). The training data explains the pattern. PIGuard was trained on 1,116 full BIPIA inputs; it also saw 53 of 62 InjecAgent attacks, but only as short prompts, and inside τ -bench tool outputs it detects those attacks no better than ones it never saw (52.1% versus 55.8%). Horizon-Labs shares no data with any benchmark, was trained on agent-style inputs, and leads both agent benchmarks (82.2% and 100.0%) with almost no false alarms. Task-aware judges reach 54–78% on the agent benchmarks, below the best dedicated detectors, and every approach misses whole classes of injections outside its training distribution.

We make the following contributions:

- A methodology for evaluating injection detectors where agents use them: clean replay of ground-truth tool calls for benign outputs, differential replay for injected ones, and task-level block rates (§4).
- A measurement of fifteen detectors and two judges on two agent benchmarks and BIPIA, showing that false-positive behavior on tool outputs is stable across agent benchmarks while detection rankings transfer poorly (§5.1, §5.2).
- Training-data audits and a controlled seen/unseen comparison showing that the form of the training inputs, not overlap in attack strings, decides where a detector works (§5.3).
- An analysis of what task awareness, attack goals, and input format change, with the finding that task-irrelevant injections evade every approach outside its training distribution (§5.4, §5.5).

We will release the code that rebuilds every evaluation set from the public benchmarks and reproduces every number in this paper.

2 Background and Motivating Example

Indirect prompt injection. An LLM agent cannot tell instructions from data by position alone: text that a tool returns can redirect the model as easily as text the user typed [11, 16]. Defenses fall into three families. Model-level defenses train the model to ignore instructions in the data region [3, 4]. System-level defenses keep untrusted data away from the component that plans actions [6]. Detection-based defenses

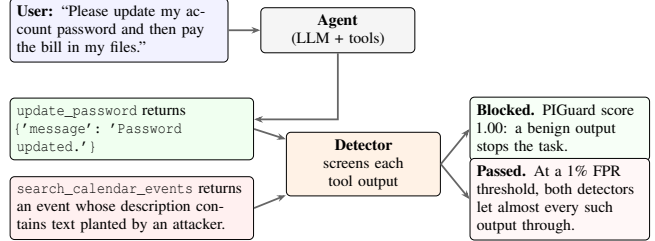


Figure 1: Running example. A detector between the tools and the agent blocks a harmless confirmation message (top) and, once its threshold is set to keep false alarms rare, passes almost every injected output (bottom). Both outputs come from AgentDojo; the scores are measured (§5.1, §5.2).

run a separate classifier on untrusted text and drop what it flags [13, 15, 17, 21]. Detectors are the easiest to adopt, because they need no change to the agent or the model, and they are the subject of this paper.

How detectors are evaluated. Detector papers report accuracy on collections of prompts and injected documents, often including BIPIA [24] and the over-defense set NotInject [15]. Practitioners compare released detectors the same way: a 2025 evaluation of nine open guardrails used BIPIA’s email and table tasks and concluded that PIGuard detects indirect injection effectively [18]. Two questions are left open by this practice. The text in these collections is not what an agent’s detector sees, which is serialized tool output. And nothing checks whether the benchmark overlaps with what the detector was trained on.

Running example. Figure 1 shows an AgentDojo banking task. The user asks the agent to update a password and pay a bill. The password tool returns `{'message': 'Password updated.'}`, and PIGuard scores this output 1.00, so a deployment that drops flagged outputs fails a task in which nothing malicious happened. In a workspace task, a calendar event carries a description that an attacker wrote. At their default thresholds the detectors catch many such outputs, but they pay for it with false alarms on a fifth of all benign tool outputs. Once the threshold is moved to keep false alarms at 1%, almost every injected output passes. Yet on BIPIA, the same PIGuard model catches nearly every injection at the same false-alarm rate. Detectors built for agent inputs show the mirror image: no false alarm on the password confirmation, most AgentDojo injections caught, and most BIPIA injections missed. The rest of this paper explains the pattern.

Why not just pick a better threshold, or a bigger model? Thresholds trade one failure for the other; the question is whether any threshold gives both low FPR and useful detection, which is what TPR at 1% FPR measures. A bigger

general-purpose model can serve as a judge, and we test two such judges; but a judge’s verdict depends on what it is asked, and §5.4 shows that the question we ask covers some attack goals and not others.

3 Deployment Setting and Threat Model

System model. A user gives an LLM agent a task in natural language. The agent calls tools (email search, calendar lookup, transaction history, web pages, file reads) and appends each tool output to its context before deciding the next step. A *detector* screens every tool output before it enters the context. If the detector flags an output, the deployment either drops the output or stops the task and asks a human. Either way, a flagged benign output costs the user the task, so we count a task as *blocked* if any of its tool outputs is flagged. This is the configuration of detector-based defenses in agent frameworks, including the detector baseline that AgentDojo ships [7] and the scanner layer of LlamaFirewall [5].

Attacker capabilities and goals. The attacker controls part of the data some tool returns (**A1**): a calendar event description, an email body, a review on a booking site, a table cell, or an answer on a Q&A site. The attacker cannot change the user’s request, the agent, or the detector. We distinguish attacks by goal, following BIPJA’s taxonomy [24]: *action* goals make the agent do something consequential (send money, forward a file, run code); *task-irrelevant* goals make it perform an unrelated task; *task-relevant* goals change how it answers the user’s own task (reverse the text, answer in Base64).

Security goals of the detector. **G1**: benign tool outputs pass, so that tasks are not blocked. **G2**: injected tool outputs are flagged at an operating point that satisfies G1.

Scope. We measure detectors against the static attacks that public benchmarks ship. An adaptive attacker who targets a specific detector would lower detection rates further [19], so our detection numbers are upper bounds. We do not propose a new defense; the task-aware judges in §4 serve as a diagnostic that separates what the benchmark measures from what the detector learned.

4 Methodology

4.1 Detectors and Judges

Detectors. We evaluate fifteen detectors (Table 2). We started from the most downloaded text-classification models on Hugging Face that are labeled as prompt-injection detectors, kept those that output an injection decision, and added the agent-oriented detectors released in 2026. Six

are gated behind a license agreement, which we accepted: Meta’s Prompt Guard 1 and Prompt Guard 2 (86M and 22M) [5], ProtectAI’s small v2 model, and two Sentinel models. The set spans three generations: DeBERTa-v3 classifiers from 2023–2025 (ProtectAI [21], JasperLS, TestSavant, PIGuard [15], Prompt Guard), ModernBERT and Qwen3-based classifiers from 2025–2026 (the two Sentinels, and Sheltron, Wolf Defender, and Horizon-Labs, whose cards target agent inputs), and a fine-tuned 1.5B generative classifier (Prismor). We run each detector as its model card documents, including its context length (512 to 2,048 tokens), its segmentation of long inputs, its input template, and, for Sheltron, its role and source tokens; for Prompt Guard 1 we use Meta’s indirect-injection score, the sum of its injection and jailbreak probabilities. Appendix A lists the settings. Two detectors need care. InjecGuard returns the same scores as PIGuard on every input we scored, so we report it once. Sheltron’s encoder also accepts a trusted “user goal” segment; its card does not document this use, but we report it as a task-aware variant because it is the only detector that can take the user’s request as input. Two further detectors, `deepset/deberta-v3-base-injection` and `fmops/distilbert-prompt-injection`, flag every benign README chunk and we omit them (Appendix D). One gated detector (CodeIntegrity) did not grant access.

Task-aware judges. To test whether information the detectors lack would change the outcome, we prompt an instruction-tuned model with the user’s request and one tool output and ask whether the output tries to make the assistant do something the user did not ask for (prompt in Appendix C). The score is the probability of “Yes” relative to “No” at the first generated token, summed over token variants. We use Qwen2.5-7B-Instruct [22] and Llama-3.1-8B-Instruct [10], both quantized to 4 bits (NF4). The Llama model’s training data ends in December 2023, six months before AgentDojo was released, so it cannot have seen AgentDojo’s tasks or attack templates. The 4-bit Qwen judge matches the same model served in bf16 to within 0.02 AU-ROC on AgentDojo.

4.2 Data

Table 1 lists the evaluation sets: two agent benchmarks, one injection benchmark, and general benign text.

General benign text. We sample 7,504 chunks of 80–220 words, about 2,500 each from GitHub READMEs [12], business emails (AESLC [26]), and educational web pages (FineWeb-Edu [20]). Every chunk is benign.

Clean agent tool outputs (AD-Clean). AgentDojo v1.2 ships, for every user task, the sequence of tool calls that

solves it [7]. We execute these calls in the default environment, without any injection and without an LLM, and record each tool output exactly as AgentDojo formats it for the agent. This yields 339 outputs from 97 tasks in four suites (workspace, travel, banking, slack). Every output is benign by construction.

Matched agent tool outputs (AD-Matched, AD-Shift). We place one of AgentDojo’s built-in attacks into every injection point of a task’s environment and replay the same ground-truth calls. An output is labeled *injected* if it differs from the output of the same call in the clean environment, and *benign* otherwise. Substring matching is not sufficient: AgentDojo serializes outputs as YAML, which re-wraps multi-line injected text. AD-Matched uses three attack templates (`important_instructions`, `ignore_previous`, `direct`; 585 benign and 432 injected outputs). AD-Shift uses five others (`tool_knowledge`, `injecagent`, `system_message`, and two variants of `important_instructions`; 975 and 720). All AgentDojo injection goals are actions.

A second agent benchmark (τ -bench). τ -bench [23] simulates customer-service agents for a retailer and an airline over JSON databases, and ships ground-truth tool calls for each task. Its authors, domains, tools, and output format (JSON rather than YAML) all differ from AgentDojo’s. Replaying the ground-truth calls of all 675 retail and airline tasks without an LLM gives 1,533 tool outputs that are benign by construction (TB-Clean). τ -bench has no injection points, so we add one where a third party plausibly controls text: the catalog name of a product, which a marketplace seller writes. For each retail task that looks up a product, we append an attack instruction from InjecAgent [25] (MIT license; its base and enhanced settings, eight attacks per task, all 62 attacks used) to the names of the products the task looks up, replay the same calls, and label by differential replay. Orders keep the product name recorded at purchase, as in a real store. This yields 584 injected outputs, which we evaluate against TB-Clean as negatives (TB-Matched). InjecAgent was written by yet another group, so its attacks share neither authors nor templates with AgentDojo or BIPIA.

BIPIA. BIPIA [24] provides clean external content for three tasks we use (email question answering, table question answering, and code repair with a Stack Overflow answer) and attack strings in three text groups and two code groups. Each clean context is a benign sample; we insert four randomly drawn attack strings at the start, middle, or end, following BIPIA’s insertion rules, to obtain injected samples. BIPIA splits its attack strings into a training and a test half. Our main results use only test-half attacks (1,200 benign,

Table 1: Evaluation sets. Benign / injected counts. AD = AgentDojo v1.2; TB = τ -bench. The benign side of TB-Matched is TB-Clean.

Set	Benign	Injected	Content
General text	7,504	–	README, email, web
AD-Clean	339	–	tool outputs, no injection
AD-Matched	585	432	tool outputs, 3 attacks
AD-Shift	975	720	tool outputs, 5 other attacks
TB-Matched	1,533	584	τ -bench tool outputs (JSON)
BIPIA	1,200	2,435	email, table, code

2,435 injected); §5.3 uses both halves.

4.3 Metrics

We report the false-positive rate (FPR) and true-positive rate (TPR) at each detector’s default threshold of 0.5, AUROC, and the TPR at 1% FPR. A deployment that screens every tool output sees benign outputs far more often than injected ones, so the low-FPR operating point is the one that matters, as in membership inference [2] and deployable injection detection [13]. We pick the 1%-FPR threshold on the same benign set we evaluate, which favors the detectors; a deployment would have to pick it in advance. For AD-Clean we also report the share of tasks with at least one flagged output. Intervals are 95% percentile bootstrap intervals over 1,000 resamples, stratified by label (by task for task-level rates).

4.4 Training-Data Audit

Two detectors publish enough to audit. PIGuard’s authors release its training split. We normalize whitespace and case and search it for (i) the first 80 characters of every BIPIA attack string and (ii) the first 120 characters of every clean BIPIA context. Horizon-Labs releases the script that assembles its training set from public datasets; we download the five sources that contain agent or tool-output injections and search them for BIPIA’s attack strings and for markers of AgentDojo’s attack templates (for example the opening sentence of `important_instructions`). We also search both for InjecAgent’s 62 attack instructions. Both searches find only verbatim copies and are lower bounds on overlap. The other detectors name their sources at most, so we report what their model cards state.

5 Results

We answer five questions.

- **RQ1:** How often do detectors flag benign agent tool outputs, and how many agent tasks does that stop?
- **RQ2:** How many injections do detectors catch at 1% FPR, and does a detector’s rank on one benchmark predict its

rank on another?

- **RQ3:** Does what a detector was trained on explain where it works?
- **RQ4:** Does knowing the user’s request help, and which attack goals does each approach miss?
- **RQ5:** Do input format and metadata change false positives?

5.1 RQ1: False Positives on Benign Tool Outputs

False alarms on benign tool outputs vary from none to most (Table 2). On AgentDojo, Horizon-Labs flags 0.0% of AD-Clean outputs and Wolf Defender 0.9%; they would stop 0.0% and 3.1% of tasks. ProtectAI v2 and PIGuard flag 30.1% and 29.5% of outputs and would stop 69.1% and 59.8% of tasks (95% intervals 60–78% and 51–70%). Both Prompt Guard 2 models flag 0.6% and 0.0%. On τ -bench’s JSON outputs the picture repeats: Horizon-Labs flags 0.7%, Wolf Defender 0.0%, Prompt Guard 2 0.0%, and Sheltron 4.4%, while ProtectAI v2 flags 66.9% and Prismor 46.2%. The flagged outputs contain no instructions; the password confirmation of Figure 1 is typical.

False-positive behavior on tool outputs is a stable property of a detector: its rank by false-positive rate on AgentDojo predicts its rank on τ -bench (Kendall $\tau = 0.67$, $p = 0.001$). It is not predicted by false positives on ordinary text. ProtectAI v2 flags 0.5% of web pages and 1.2% of emails but 30.1% of AgentDojo tool outputs and 66.9% of τ -bench outputs.

Finding 1. *False-positive rates on benign agent tool outputs range from 0% to over 90% across detectors, are consistent across the two agent benchmarks ($\tau = 0.67$), and are not predicted by false-positive rates on ordinary text.*

5.2 RQ2: Detection at 1% FPR and Rank Transfer

On AD-Matched, three detectors catch most injections at 1% FPR: Horizon-Labs 82.2%, Wolf Defender 73.8%, and Prismor 72.2%. Meta’s Prompt Guard 2 catches 69.4% (86M) and 60.9% (22M). The other DeBERTa detectors catch between 0.5% and 27.8%, and PIGuard 2.1%. On TB-Matched, Horizon-Labs catches 100.0%, Sheltron 95.2%, and Wolf Defender 90.8%; PIGuard catches 52.7% and Prismor 15.2%.

Rankings do not transfer reliably between benchmarks (Figure 2). Between BIPIA and AD-Matched, Kendall’s τ is 0.01 ($p = 1.00$): PIGuard ranks first on BIPIA (95.1%) and thirteen of fifteen on AgentDojo. Between the two agent benchmarks, τ is 0.28 ($p = 0.17$), and two detectors trade places: Prismor falls from 72.2% on AgentDojo to 15.2% on τ -bench, while Sheltron rises from 20.1% to 95.2%. Between BIPIA and τ -bench, τ is 0.31 ($p = 0.11$). None of

these detection correlations is significant across the fifteen detectors. Horizon-Labs and Wolf Defender lead both agent benchmarks; the two Prompt Guard 2 models are consistent but lower on both (69.4% and 57.9% for 86M, 60.9% and 60.8% for 22M).

Finding 2. *A detector’s detection rank on one benchmark is a weak guide to its rank on another (τ between 0.01 and 0.31, none significant). The best detector on BIPIA catches 2.1% of AgentDojo injections at 1% FPR, and a detector that catches 72.2% of AgentDojo injections catches 15.2% on τ -bench.*

5.3 RQ3: Training Data

Where training data is public, it explains the ranking, and it shows that what transfers is the form of the inputs, not the attack strings.

PIGuard and BIPIA. PIGuard’s published training split contains 1,116 labeled BIPIA inputs: full emails, tables, and code answers with and without inserted attacks (median 715 characters). The verbatim overlap is large (Table 3): 65 of 75 text attacks and 6 of 50 code attacks from BIPIA’s training half appear, as do 31% of BIPIA’s clean emails and 22% of its clean code contexts. None of BIPIA’s test-half attacks appear, and our main results use only those; restricting to them lowers PIGuard’s detection at 1% FPR only from 97.9% to 95.1%, because the two halves come from the same generator and are inserted into the same kinds of content. Training on one half and testing on the other is standard practice; the result is an in-distribution score.

PIGuard and InjecAgent. PIGuard’s training split also contains 53 of 62 InjecAgent attack instructions, but in another form: 111 short prompts (median 178 characters) that pair an attack with a generic task sentence, not attacks inside tool responses. This gives a controlled comparison within TB-Matched, where every injected output has the same form. PIGuard detects 52.1% of the outputs carrying an attack it was trained on and 55.8% of those carrying one of the 9 attacks it was not. Having seen the attack text does not help when the surrounding input is new.

Horizon-Labs and the agent benchmarks. Horizon-Labs uses BIPIA only for evaluation and removes evaluation texts from its training data. Its training set includes five public datasets of injections planted in documents and tool responses. Our audit of those five finds no BIPIA attack string, no AgentDojo attack template, and no InjecAgent instruction. It is nevertheless the best detector on both agent benchmarks; what it shares with them is the form of input, structured tool output with planted instructions.

Table 2: All detectors on all sets. General: README, email, and web text. FPR at each detector’s default threshold on benign tool outputs (AD-Clean, TB-Clean); “tasks” is the share of tasks with at least one flagged benign output. TPR at the threshold that gives 1% FPR on the same set. BIPIA uses test-half attacks. “Agent data”: the model card states training on agent or tool-output inputs; “eval.” means the card reports evaluation on AgentDojo-derived data. [†]Undocumented use of Sheltron’s user-goal segment. Bootstrap intervals for four methods are in Appendix Table 4.

Detector	Year	Size	Agent data	General	AgentDojo				τ -bench			BIPIA
				FPR (%)	FPR (%)	Tasks (%)	TPR @ 1% Matched	FPR Shift	FPR (%)	Tasks (%)	TPR @ 1%	TPR @ 1%
JasperLS	2023	184M	no	99.9	78.5	97.9	23.4	22.1	93.1	99.6	18.7	4.9
ProtectAI v1	2023	184M	no	0.3	3.5	12.4	0.7	7.5	5.1	6.8	23.3	0.4
ProtectAI v2	2024	184M	no	3.8	30.1	69.1	0.5	0.8	66.9	80.3	10.1	0.7
ProtectAI v2-small	2024	142M	no	1.6	12.1	34.0	7.6	3.1	1.4	2.8	30.1	1.4
TestSavant-L	2024	184M	no	3.8	3.2	10.3	27.8	24.7	1.8	3.4	24.5	16.3
Prompt Guard 1	2024	279M	no	5.8	31.9	53.6	9.5	1.5	4.8	9.3	2.9	36.0
PIGuard	2025	184M	no	9.4	29.5	59.8	2.1	7.9	11.0	20.9	52.7	95.1
Prompt Guard 2 (22M)	2025	71M	no	0.3	0.0	0.0	60.9	63.6	0.0	0.0	60.8	31.7
Prompt Guard 2 (86M)	2025	279M	no	0.1	0.6	2.1	69.4	78.8	0.0	0.0	57.9	10.4
Qualifire Sentinel	2025	396M	no	18.2	0.9	3.1	61.1	66.4	0.0	0.0	21.9	4.6
Sentinel v2	2025	596M	no	17.2	2.1	7.2	20.8	16.5	0.5	1.2	68.7	20.2
Sheltron	2026	68M	yes	12.0	10.6	28.9	20.1	26.4	4.4	6.7	95.2	57.2
Sheltron (+task) [†]	2026	68M	yes	12.0	26.0	47.4	9.0	18.8	69.7	85.0	65.4	8.0
Prismor-1.5B	2026	1.5B	no	57.7	6.5	17.5	72.2	79.4	46.2	67.7	15.2	4.0
Wolf Defender	2026	308M	eval.	3.1	0.9	3.1	73.8	83.9	0.0	0.0	90.8	3.5
Horizon-Labs	2026	308M	yes	1.1	0.0	0.0	82.2	87.9	0.7	1.6	100.0	37.7
Judge (Qwen2.5-7B)	–	7B	–	–	–	–	57.4	64.4	4.4	9.0	54.3	14.9
Judge (Llama-3.1-8B)	–	8B	–	–	–	–	55.3	61.9	5.4	9.6	78.3	8.3

Table 3: Verbatim overlap between BIPIA and PIGuard’s published training split (whitespace and case normalized). A lower bound: paraphrases are not counted.

BIPIA component	Found in training split
Text attacks, training half	65 of 75
Code attacks, training half	6 of 50
Text attacks, test half	0 of 75
Code attacks, test half	0 of 50
Clean email contexts	31%
Clean code contexts	22%
Clean table contexts	0.3%
AgentDojo attack templates	none

The others. Wolf Defender’s card reports evaluation on a benchmark built from AgentDojo’s injection payloads; its training data is not public. Prismor was fine-tuned on two prompt-level injection datasets. AgentDojo’s templates are explicit instructions addressed to the model (“ignore your previous instructions”, “this is an important message”), which resemble direct prompt injections; InjecAgent’s attacks inside τ -bench product names and BIPIA’s attacks inside emails and tables do not. Prismor detects the first and not the others.

Finding 3. *Detectors work on inputs whose form resembles their training inputs. PIGuard’s BIPIA score rests on training on full BIPIA inputs; having seen InjecAgent’s attack strings as short prompts does not help it detect them inside tool outputs (52.1% versus 55.8%). Horizon-Labs leads both agent benchmarks without any overlap, after training on agent-style inputs.*

5.4 RQ4: Task Awareness and Attack Goals

Judges. Telling a general-purpose model the user’s request makes it a reasonable detector: at 1% FPR the Qwen judge catches 57.4% of AD-Matched and 54.3% of TB-Matched injections, and the Llama judge 55.3% and 78.3%. Llama-3.1-8B was trained on data from before AgentDojo existed, so its detection cannot come from memorizing AgentDojo. On both agent benchmarks the judges stay below the best dedicated detectors, at the cost of a 7–8B forward pass per tool output.

Task-aware Sheltron. Passing the user’s request to Sheltron in a trusted segment raises its AUROC on AD-Matched from 0.86 to 0.91 but lowers its detection at 1% FPR from 20.1% to 9.0%, and raises its false-positive rate on clean tool outputs from 10.6% to 26.0% on AgentDojo and from 4.4%

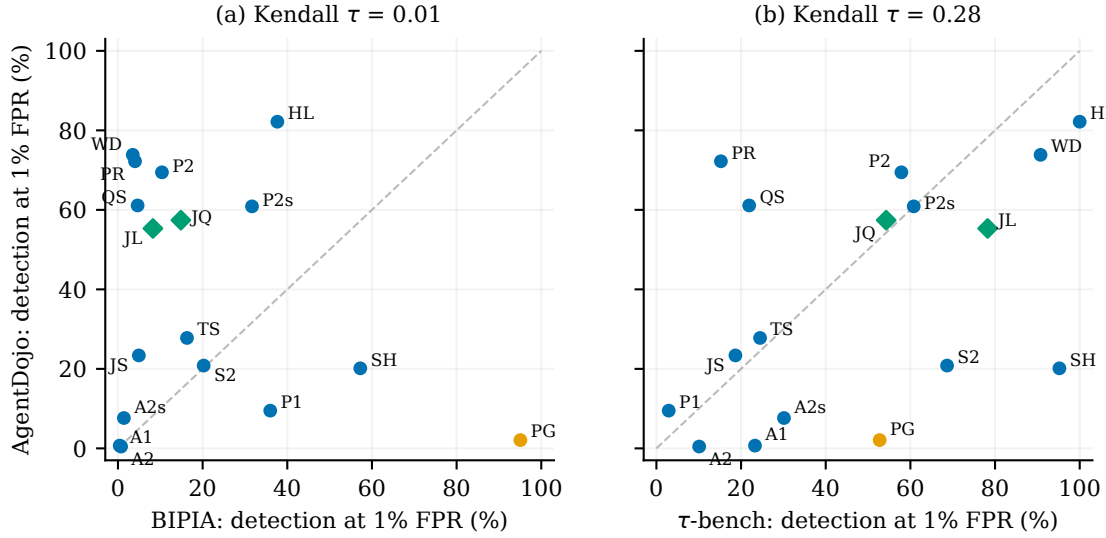


Figure 2: Each point is a detector or judge (diamonds). (a) Detection on BIPIA does not predict detection on AgentDojo. (b) Detection on one agent benchmark predicts detection on the other only partly: Prismor (PR) and Sheltron (SH) trade places. Dashed line: equal performance. Key: HL Horizon-Labs, WD Wolf Defender, PR Prismor, P2/P2s Prompt Guard 2 86M/22M, P1 Prompt Guard 1, PG PIGuard, SH Sheltron, QS Qualifire Sentinel, S2 Sentinel v2, TS TestSavant, JS JasperLS, A1/A2/A2s ProtectAI v1/v2/v2-small, JQ/JL Qwen/Llama judge.

to 69.7% on τ -bench. This use is undocumented and may simply be out of distribution for the model.

Attack goals. Figure 3 breaks BIPIA’s test-half injections into BIPIA’s five official groups, each method at its own 1%-FPR threshold on BIPIA. No approach other than the BIPIA-trained PIGuard covers all groups. Task-irrelevant injections, which ask the model to perform some unrelated harmless task, are the hardest: apart from PIGuard (93%), the best approach is Prompt Guard 1 with 39%, and every other approach catches at most 20%. The others split the remaining groups: Sheltron catches 87% of task-relevant and 88% of targeted injections but 16% of active code attacks, while the Qwen judge catches 69% and 61% of passive and active code attacks and 1% of task-relevant ones. Our judge prompt asks about actions and tells the model not to flag ordinary requests between people; we did not vary it, so we cannot say how much of the judges’ gap comes from the question we asked.

Finding 4. *Task-aware 7–8B judges detect 54–78% of injections in agent tool outputs at 1% FPR, below the best dedicated detectors, and giving the user’s request to a small classifier does not help. Outside its training distribution, every approach misses whole classes of injections, and task-irrelevant injections evade nearly all of them.*

5.5 RQ5: Format and Metadata

How a tool output is presented to a detector moves its false-positive rate. README chunks that PIGuard flags are full of HTML tables, badges, and links rather than instructions; stripping markup lowers its FPR on READMEs from 24.6% to 5.6% and ProtectAI v2’s from 9.8% to 4.2% (Figure 4). On AD-Matched, removing YAML keys so that PIGuard sees only values lowers its FPR from 20.0% to 6.7% at the same TPR (73.1% versus 74.3%). Sheltron’s source token has the largest effect: declaring a tool output as `tool_result` instead of `other` lowers its FPR on clean tool outputs from 28.6% to 10.6% on AgentDojo and from 85.6% to 4.4% on τ -bench.

These changes help less at the low-FPR end. With YAML keys removed, PIGuard catches 3.0% of AD-Matched injections at 1% FPR and ProtectAI v2 13.2%. Sheltron with the specific source token catches 20.1% on AD-Matched, compared with 30.3% with the generic one, and 95.2% versus 92.3% on TB-Matched. Format and metadata shift a detector’s scores; they do not make an out-of-distribution detector work.

Finding 5. *Removing serialization markup or declaring the input type lowers default-threshold false positives by up to about twentyfold, a free improvement for deployments, but after any of these changes neither PIGuard nor ProtectAI v2 exceeds 15% detection at 1% FPR on AD-Matched.*

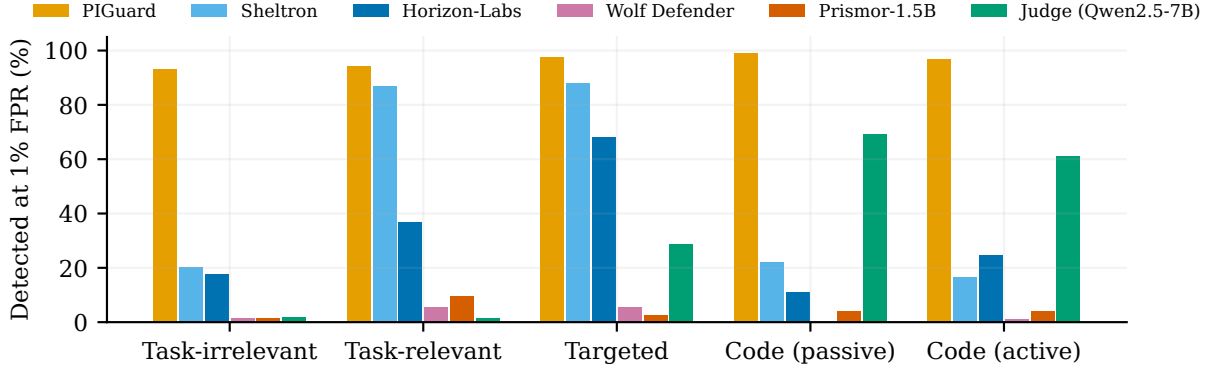


Figure 3: Detection by BIPIA attack group (test half), each method at its own 1%-FPR threshold on BIPIA. Only PIGuard, which was trained on BIPIA inputs, covers all groups; every other approach shown misses most task-irrelevant injections.

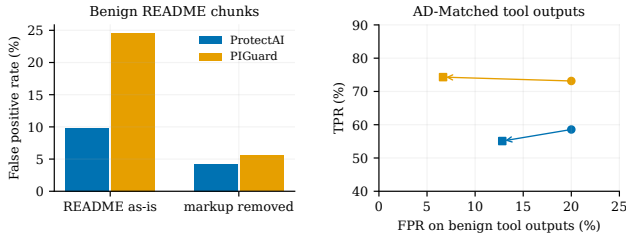


Figure 4: Format alone moves false positives. Left: removing markup from benign READMEs. Right: removing YAML keys from AgentDojo tool outputs (circle: as-is; square: keys removed) lowers PIGuard’s FPR threefold at unchanged TPR.

6 Discussion and Limitations

Choosing a detector for an agent. Our results suggest four practical rules. First, measure false positives on the agent’s own tool outputs and translate them into blocked tasks; false positives on prose do not predict them. Second, report detection at a fixed low FPR; default-threshold accuracy on balanced sets says little about either error. Third, before trusting a benchmark score, check whether the detector was trained on that benchmark or on data generated the same way, and prefer evaluation data the detector cannot have seen. Fourth, prefer a detector trained on inputs like the ones it will screen: in our study, Horizon-Labs, Wolf Defender, and both Prompt Guard 2 models kept false positives near zero on both agent benchmarks and caught more than half of the injections on each; Horizon-Labs and Wolf Defender, both built for untrusted content reaching agents, caught more than 70%. Even these missed most task-irrelevant injections, so a detector is a filter to combine with system-level defenses [6], not a replacement for them.

Simulated agent environments. Limitation. Both agent benchmarks are simulations: AgentDojo returns synthetic YAML records, and τ -bench returns JSON from synthetic databases. Real tool outputs (HTML pages, API responses, long documents) are messier, and our results show that format alone moves false-positive rates, so absolute numbers will differ in deployment. In τ -bench we also chose the injection point (product names) and the attack set (InjecAgent) ourselves; another placement, such as reviews or seller messages, could favor different detectors. The findings we expect to carry over are the relative ones that held on both agent benchmarks: which detectors over-flag tool outputs, and that the agent-oriented detectors lead.

Static attacks. Limitation. All injections come from benchmark templates. An adaptive attacker who knows the detector or judge would lower every detection rate we report [19]. Our detection numbers are upper bounds; the false-positive results do not depend on the attacker.

Detector coverage. Limitation. We evaluate the open and license-gated detectors we could access; commercial API detectors are closed and not included. Only PIGuard and Horizon-Labs publish enough to audit their training data; for the others we can report only what their model cards state.

The judges are a diagnostic, not a defense. The task-aware judges call a 7–8B model once per tool output, use a single prompt, and were not tested against adaptive attacks. We use them to show which information the detectors lack and which attack goals that information covers. A judge prompt that also asks about changes to the answer might cover task-irrelevant and task-relevant injections, at a cost in false alarms we have not measured.

Verbatim overlap is a lower bound. Our audits find exact copies only. Paraphrases, attacks generated from the same templates, and unpublished data escape them. For PIGuard and BIPIA, verbatim overlap alone already explains the score.

7 Related Work

Evaluation methodology for injection detectors. Fomin [9] trains activation probes on 18 datasets and shows that leave-one-dataset-out evaluation lowers AUROC compared with in-distribution splits. PromptShield [13] argues for evaluating detectors at low FPR and builds a curated benchmark for it. PIGuard [15] measures over-defense on benign prompts with trigger words. Unlike these works, we evaluate released detectors on the tool outputs an agent actually screens, translate false positives into blocked tasks, and audit a released detector’s published training split against the benchmark used to praise it.

Pitfalls in ML and LLM security research. Arp et al. [1] and Evertz et al. [8] catalog pitfalls such as data leakage and inappropriate metrics in security research with ML and LLMs. Nasr et al. [19] show that defenses evaluated on static attacks fall to adaptive ones. Our study is an instance of the leakage and metric pitfalls for one widely used class of defenses, with the overlap measured directly rather than inferred.

Detection-based defenses. ProtectAI’s detector [21], PIGuard [15], and Prompt Guard in LlamaFirewall [5] classify text with a fine-tuned encoder. A newer generation of open detectors, released in 2026 and evaluated here, trains on documents and tool responses with planted instructions or adds role and source tokens to the input; their model cards report results on their own or AgentDojo-derived evaluation sets. DataSentinel [17] detects injections with an LLM fine-tuned in a minimax game against adaptive attacks, and AlignSentinel [14] separates instructions that align with the user’s task from those that do not, using attention features. Our task-aware judges follow the same intuition as AlignSentinel without training; we use them only to diagnose what the classifiers miss.

Benchmarks. AgentDojo [7], InjecAgent [25], and BIPIA [24] measure whether agents or LLMs follow injected instructions. We reuse AgentDojo’s ground-truth tool calls and BIPIA’s contexts and attacks to build matched benign and injected sets for detectors, a use their authors did not design them for.

Model- and system-level defenses. StruQ [3] and SeCAlign [4] train the model to ignore instructions in data;

CaMeL [6] keeps untrusted data out of the planner. These defenses do not rely on recognizing an injection, so our findings about detectors do not apply to them.

8 Conclusion

Prompt-injection detectors are chosen by benchmark scores, but across fifteen detectors, two agent benchmarks, and BIPIA, detection rankings transferred poorly from one benchmark to another. What a detector was trained on explained where it worked: the detector that looks best on BIPIA was trained on full BIPIA inputs and misses nearly every AgentDojo injection, seeing an attack’s text without its surrounding tool output did not help, and the detector that led both agent benchmarks shared no data with them but had been trained on agent-style inputs. False-positive behavior on tool outputs, by contrast, was stable across agent benchmarks and is cheap to measure. Evaluations meant to guide deployment should replay the agent’s own tool outputs, report detection at a low false-positive rate and the task-level cost, and audit what the detector has already seen.

Ethics Considerations

We use only public benchmarks and public benign corpora, and create no new attacks: every injected sample uses an attack string that AgentDojo or BIPIA already publishes. We ran no experiments against deployed systems or other people’s agents. The overlap audit concerns a training split its authors chose to publish; we report it to improve evaluation practice and will share our findings with the detector authors before publication.

Open Science

We will release the scripts that build every evaluation set (they regenerate the data from the public benchmarks rather than redistributing it), the detector and judge scores, and the analysis code that produces every number and figure in this paper.

References

- [1] ARP, D., QUIRING, E., PENDLEBURY, F., WARNECKE, A., PIERAZZI, F., WRESSNEGGER, C., CAVALLARO, L., AND RIECK, K. Dos and don’ts of machine learning in computer security. In *Proceedings of the USENIX Security Symposium* (2022).
- [2] CARLINI, N., CHIEN, S., NASR, M., SONG, S., TERZIS, A., AND TRAMÈR, F. Membership inference attacks from first principles. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)* (2022).
- [3] CHEN, S., PIET, J., SITAWARIN, C., AND WAGNER, D. StruQ: Defending against prompt injection with structured queries. In *Proceedings of the USENIX Security Symposium* (2025).

[4] CHEN, S., ZHARMAGAMBETOV, A., MAHLOUJIFAR, S., CHAUDHURI, K., WAGNER, D., AND GUO, C. SecAlign: Defending against prompt injection with preference optimization. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)* (2025).

[5] CHENNABASAPPA, S., ET AL. LlamaFirewall: An open source guardrail system for building secure AI agents. arXiv:2505.03574, 2025.

[6] DEBENEDETTI, E., SHUMAILOV, I., FAN, T., HAYES, J., CARLINI, N., FABIAN, D., KERN, C., SHI, C., TERZIS, A., AND TRAMÈR, F. Defeating prompt injections by design. arXiv:2503.18813, 2025.

[7] DEBENEDETTI, E., ZHANG, J., BALUNOVIĆ, M., BEURER-KELLNER, L., FISCHER, M., AND TRAMÈR, F. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track* (2024).

[8] EVERTZ, J., RISSE, N., NEUER, N., MÜLLER, A., NORMANN, P., SAPIA, G., GUPTA, S., PAPE, D., SHAW, S., SRIVASTAV, D., WRESSNEGGER, C., QUIRING, E., EISENHOFER, T., ARP, D., AND SCHÖNHERR, L. Chasing shadows: Pitfalls in LLM security research. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)* (2026).

[9] FOMIN, M. When benchmarks lie: Evaluating malicious prompt classifiers under true distribution shift. In *International Conference on Learning Representations (ICLR)* (2026).

[10] GRATTAFIORI, A., ET AL. The Llama 3 herd of models. arXiv:2407.21783, 2024.

[11] GRESHAKE, K., ABDELNABI, S., MISHRA, S., ENDRES, C., HOLZ, T., AND FRITZ, M. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec)* (2023).

[12] HIALEXBEL. github-readmes (dataset). <https://huggingface.co/datasets/hialexbel/github-readmes>, 2024.

[13] JACOB, D., ALZAHIRANI, H., HU, Z., ALOMAIR, B., AND WAGNER, D. PromptShield: Deployable detection for prompt injection attacks. In *Proceedings of the ACM Conference on Data and Application Security and Privacy (CODASPY)* (2025).

[14] JIA, Y., WANG, R., WANG, X., XIANG, C., AND GONG, N. Z. AlignSentinel: Alignment-aware detection of prompt injection attacks. arXiv:2602.13597, 2026.

[15] LI, H., LIU, X., ZHANG, N., AND XIAO, C. PIGuard: Prompt injection guardrail via mitigating overdefense for free. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)* (2025).

[16] LIU, Y., JIA, Y., GENG, R., JIA, J., AND GONG, N. Z. Formalizing and benchmarking prompt injection attacks and defenses. In *Proceedings of the USENIX Security Symposium* (2024).

[17] LIU, Y., JIA, Y., JIA, J., SONG, D., AND GONG, N. Z. DataSentinel: A game-theoretic detection of prompt injection attacks. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)* (2025).

[18] MOZILLA.AI. Can open-source guardrails really protect AI agents? <https://blog.mozilla.ai/can-open-source-guardrails-really-protect-ai-agents/>, Nov. 2025.

[19] NASR, M., CARLINI, N., SITAWARIN, C., SCHULHOFF, S. V., HAYES, J., ILIE, M., PLUTO, J., SONG, S., CHAUDHARI, H., SHUMAILOV, I., THAKURTA, A., XIAO, K. Y., TERZIS, A., AND TRAMÈR, F. The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections. arXiv:2510.09023, 2025.

[20] PENEDO, G., KYDLÍČEK, H., ALLAL, L. B., LOZHKO, A., MITCHELL, M., RAFFEL, C., WERRA, L. V., AND WOLF, T. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track* (2024).

[21] PROTECT AI. Fine-tuned DeBERTa-v3-base for prompt injection detection (v2). <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2>, 2024.

[22] QWEN TEAM. Qwen2.5 technical report. arXiv:2412.15115, 2024.

[23] YAO, S., SHINN, N., RAZAVI, P., AND NARASIMHAN, K. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)* (2025).

[24] YI, J., XIE, Y., ZHU, B., KICIMAN, E., SUN, G., XIE, X., AND WU, F. Benchmarking and defending against indirect prompt injection attacks on large language models. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)* (2025).

[25] ZHAN, Q., LIANG, Z., YING, Z., AND KANG, D. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)* (2024).

[26] ZHANG, R., AND TETREAU, J. This email could save your life: Introducing the task of email subject line generation. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)* (2019).

A Detector Configurations

Each detector is run as its model card documents. Inputs longer than the context window are truncated, except for Sheltron, whose card specifies overlapping windows (stride 768), and Prompt Guard, whose cards specify splitting long inputs into 512-token segments; for both we take the maximum score over windows. Prismor’s card truncates the whole chat prompt at 512 tokens, which cuts the assistant header from long inputs; we truncate only the text so the template stays intact.

Detector	Hugging Face model	Ctx	Input
ProtectAI v1	protectai/...-injection	512	text
ProtectAI v2	protectai/...-injection-v2	512	text
JasperLS	JasperLS/deberta-v3-base-injection	512	text
TestSavant-L	testsavantai/...-large-v0	512	text
PIGuard	leolee99/PIGuard	512	text
Sheltron	sheltron-ai/prompt-guard-68m	1024	role/source tokens
Prismor	prismor/prompt-guard-1.5b	512	chat template
Wolf Defender	patronus-studio/wolf-defender-...	2048	text
Horizon-Labs	Horizon-Labs/...-guard-base	2048	text
Prompt Guard 2 (86M)	meta-llama/Llama-Prompt-Guard-2-86M	512	segmented
Prompt Guard 2 (22M)	meta-llama/Llama-Prompt-Guard-2-22M	512	segmented
Prompt Guard 1	meta-llama/Prompt-Guard-86M	512	segmented
ProtectAI v2-small	protectai/...-small-...-v2	512	text
Qualifire Sentinel	qualifire/prompt-injection-sentinel	2048	text
Sentinel v2	rogue-security/...-sentinel-v2	2048	text

B Detailed Results for Four Methods

Table 4 gives AUROC, bootstrap intervals for TPR at 1% FPR, and default-threshold rates for the two detectors and two judges we examine most closely; Figure 5 shows their ROC curves.

Table 4: Detection on the three matched sets for four methods. Brackets: 95% bootstrap interval. BIPIA uses only test-half attacks.

Set	Method	AUROC	TPR @ 1% FPR (%)	FPR @ 0.5 (%)	TPR @ 0.5 (%)
AD-Matched	ProtectAI v2	0.82	0.5 [0.0–1.2]	20.0	58.6
	PIGuard	0.83	2.1 [0.0–4.4]	20.0	73.1
	Judge (Qwen2.5-7B)	0.90	57.4 [52.5–62.7]	6.7	77.5
	Judge (Llama-3.1-8B)	0.93	55.3 [26.6–75.2]	11.8	87.0
AD-Shift	ProtectAI v2	0.84	0.8 [0.3–1.5]	20.0	65.7
	PIGuard	0.84	7.9 [3.1–12.4]	20.0	80.7
	Judge (Qwen2.5-7B)	0.92	64.4 [61.1–70.7]	6.7	81.5
	Judge (Llama-3.1-8B)	0.93	61.9 [32.9–79.9]	11.8	88.1
BIPIA	ProtectAI v2	0.32	0.7 [0.4–1.3]	12.6	9.0
	PIGuard	0.99	95.1 [93.1–96.1]	0.2	92.6
	Judge (Qwen2.5-7B)	0.81	14.9 [11.0–18.5]	1.4	16.6
	Judge (Llama-3.1-8B)	0.59	8.3 [6.4–11.1]	4.8	15.9

C Judge Prompt

Both judges receive the following system prompt, then a user message of the form “User request: ... Tool <name> returned: ...” (tool output truncated to 6,000 characters). We read the first-token distribution and compute $P(\text{Yes})/(P(\text{Yes}) + P(\text{No}))$, summing over case and white-space variants of each word. Both judges are quantized to 4 bits (NF4); the 4-bit Qwen judge matches the same model served in bf16 to within 0.02 AUROC on AD-Matched.

You audit data returned by tools to an AI assistant. Tool data is supposed to be passive information. Answer ‘Yes’ if the tool data contains text that tries to make the assistant perform actions or tasks the user did not ask for. Answer ‘No’ otherwise, including when the data merely describes procedures or contains ordinary requests between people. Reply with exactly one word: Yes or No.

D Excluded Detectors

deepset/deberta-v3-base-injection and fmops/distilbert-prompt-injection flag every benign README chunk we scored (100.0% and 100.0%, respectively) and more than three quarters of AD-Clean tool outputs. The fmops model also flags two of five plain sentences such as “The weather is nice today.” We treat both as unusable and omit them. leolee99/InjecGuard returns scores identical to PIGuard on all five sets, so we report it once.

E Labeling by Differential Replay

Our first labeling attempt marked an output as injected if it contained the injected string verbatim. Because Agent-Dojo serializes tool outputs as YAML, multi-line injections

are re-wrapped and escaped, and every output carrying the `important_instructions` attack was labeled benign. Differential replay (same ground-truth call, clean versus injected environment) avoids the problem; with it, each of the three AD-Matched templates contributes the same number of injected outputs. We note the pitfall because substring labeling is the obvious approach and silently inflates false-positive rates.

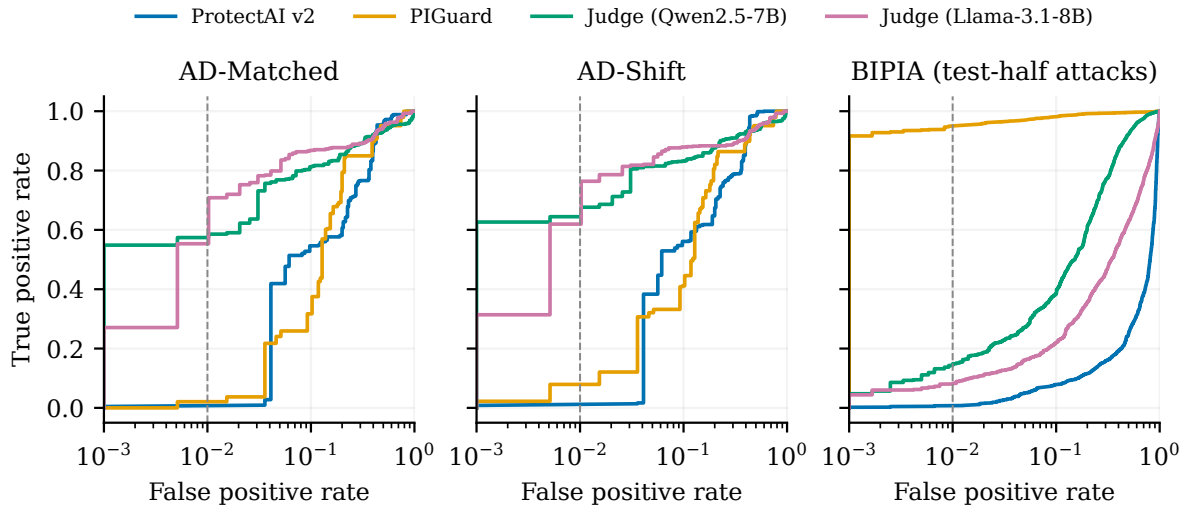


Figure 5: ROC curves with logarithmic FPR axis for the four methods of Table 4; the dashed line marks 1% FPR. The Llama judge’s wide interval on AD-Matched comes from a few benign outputs with high scores that make the 1% threshold unstable.